

BÁO CÁO MÔN HỌC

LẬP TRÌNH JAVA WEB SERVICE

ĐỀ TÀI: HỆ THỐNG QUẢN LÝ BỆNH VIỆN
(HOSPITAL MANAGEMENT SYSTEM)

Sinh viên thực hiện	Lê Trung Đông
Giảng viên hướng dẫn	Bùi Thanh Hải
Môn học	IT211 - Java Web Service

I. GIỚI THIỆU DỰ ÁN

1.1. Tổng quan hệ thống

Hệ thống Quản lý Bệnh viện được xây dựng dưới dạng ứng dụng Backend không trạng thái (Stateless Backend) cung cấp các RESTful API. Hệ thống sử dụng Spring Boot Framework, áp dụng kiến trúc phân lớp chuẩn Controller → Service → Repository → Entity. Các ứng dụng phía Client (Web hoặc Mobile) sẽ tiêu thụ các API này để vận hành nghiệp vụ của bệnh viện.

1.2. Công nghệ sử dụng

Công nghệ	Mô tả
Spring Boot 3.4.1	Framework chính, quản lý dependency injection và REST API
Spring Security + JWT	Xác thực và phân quyền, token-based authentication
Spring Data JPA + MySQL	ORM và quản lý dữ liệu với MySQL database
Spring AOP	Logging tự động qua Aspect-Oriented Programming
Cloudinary	Lưu trữ file hồ sơ bệnh án trên đám mây

1.3. Danh sách chức năng

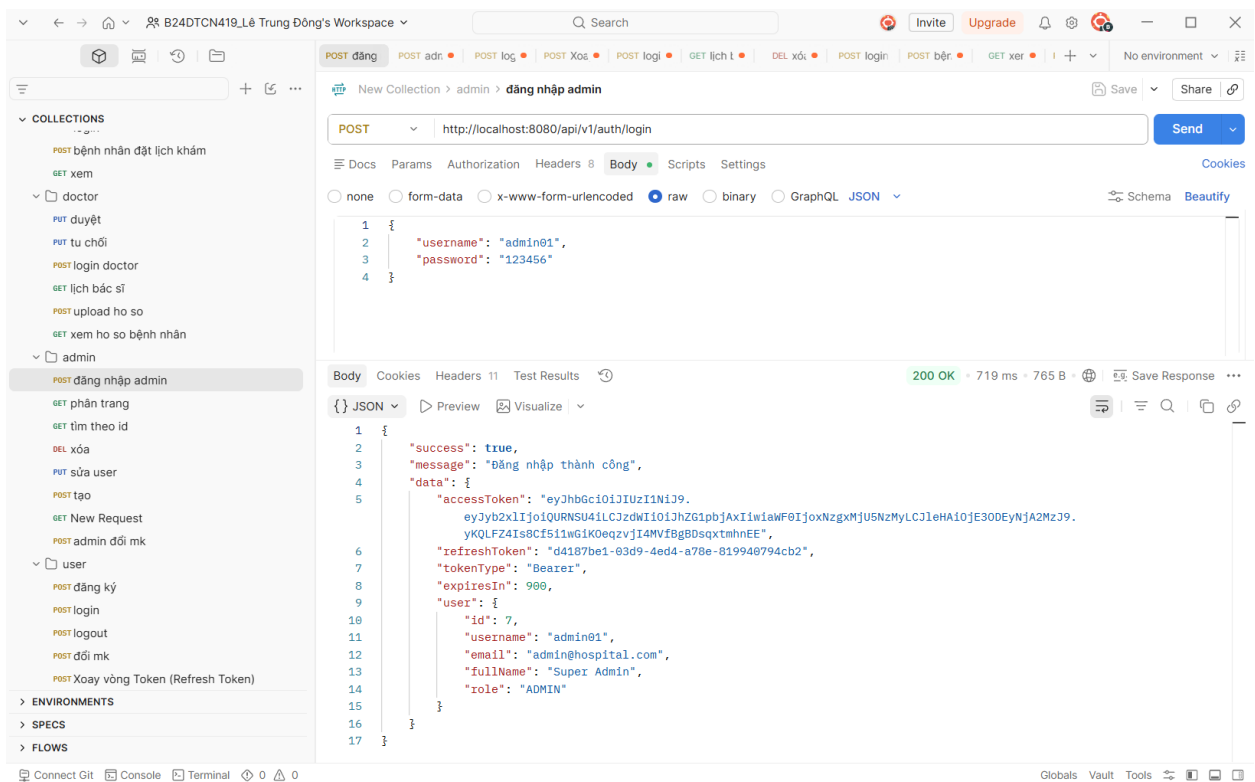
Mã FR	Tên chức năng	Phân quyền
FR-01	Đăng nhập hệ thống (Cấp phát JWT)	Public
FR-02	Xoay vòng Token (Refresh Token)	Public
FR-03	Đăng xuất (Revoke Token)	Authenticated
FR-04	Đăng ký tài khoản Bệnh	Public

	nhân mới	
FR-05	Quản lý Người dùng (CRUD, Tìm kiếm, Phân trang)	Admin
FR-06	Đặt lịch khám bệnh	Patient
FR-07	Xem lịch sử khám bệnh cá nhân	Patient
FR-08	Phê duyệt / Từ chối lịch khám	Admin, Doctor
FR-09	Tải lên Hồ sơ bệnh án (Medical Record)	Doctor
FR-10	Đổi mật khẩu	Authenticated
FR-11	Ghi log thời gian thực hiện (AF1)	Hệ thống (AOP)
FR-12	Cài đặt Unit Test (AF2)	Kiểm thử

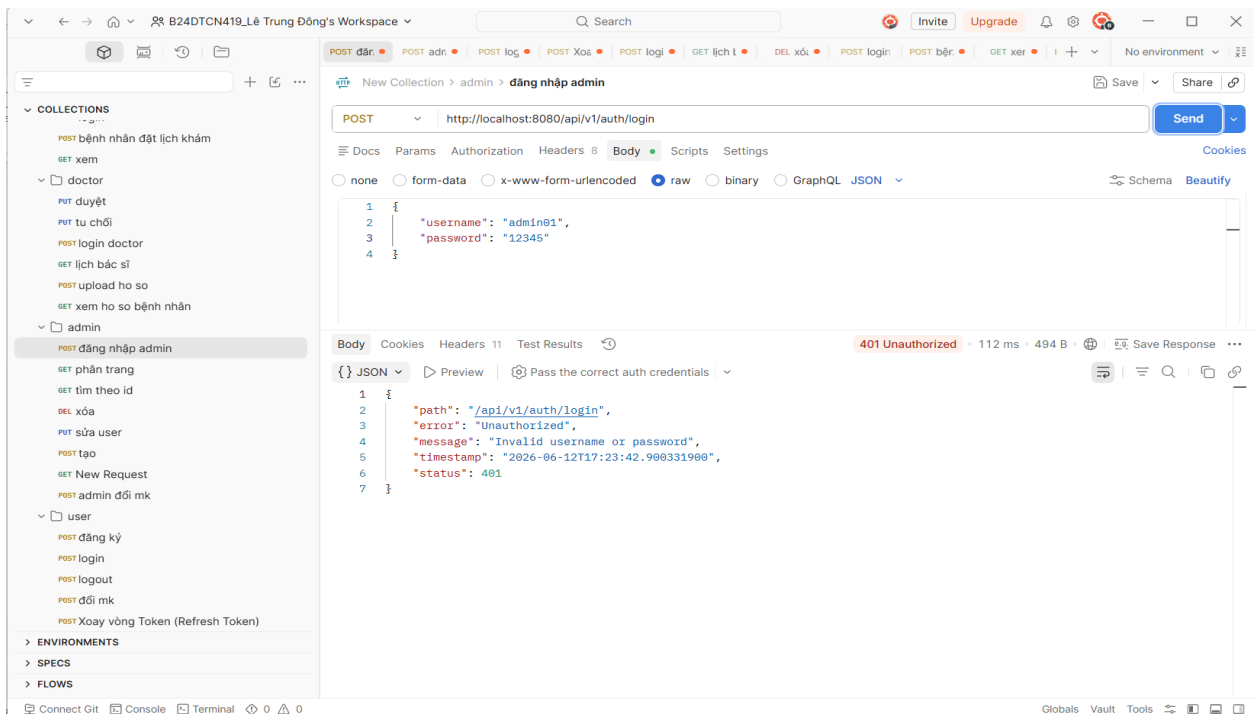
II. LƯỒNG XỬ LÝ CÁC CHỨC NĂNG

FR-01 — Đăng nhập hệ thống (Cấp phát JWT)

Bước	Mô tả xử lý
1. Xuất phát	Bất kỳ người dùng nào (Admin/Doctor/Patient) muốn đăng nhập vào hệ thống.
2. Gọi API	POST /api/v1/auth/login — Body JSON: { username, password }. Endpoint này là Public, không cần token.
3. Xác thực	AuthController nhận @Valid @RequestBody LoginRequest — Spring Validation kiểm tra username và password không trống. Nếu thiếu trả về HTTP 400 Bad Request.
4. Xử lý Service	AuthController gọi AuthServiceImpl.login(): (1) AuthenticationManager.authenticate() so sánh password BCrypt — sai ném 401. (2) Thu hồi toàn bộ RefreshToken cũ. (3) JwtService.generateAccessToken() tạo JWT 15 phút. (4) Tạo RefreshToken UUID lưu DB 7 ngày.
5. Repository	UserRepository.findByUsername() lấy thông tin user. RefreshTokenRepository.revokeAllUserTokens() thu hồi token cũ. RefreshTokenRepository.save() lưu token mới.
6. Kết quả	HTTP 200 OK + { accessToken, refreshToken, tokenType: 'Bearer', expiresIn, user: { id, username, email, role } }



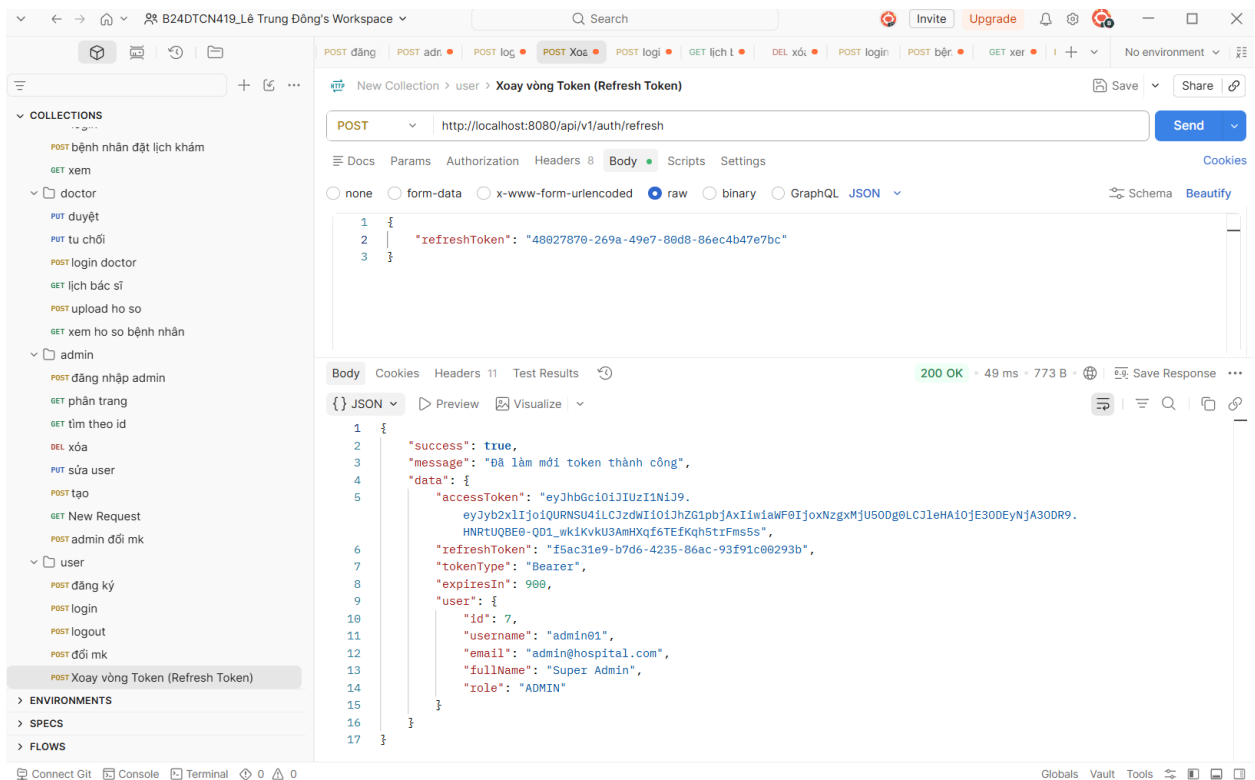
1.1 Kết quả POST /api/v1/auth/login thành công — trả về accessToken và refreshToken



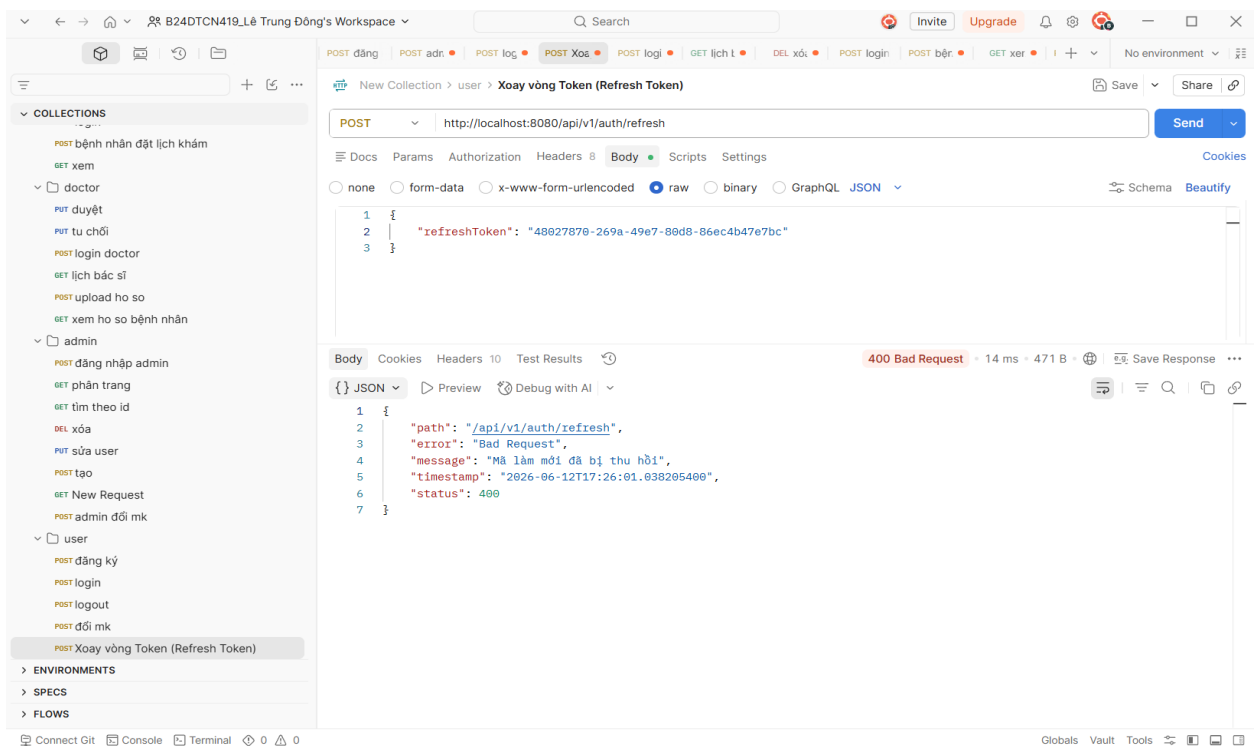
1.2 Kết quả POST /api/v1/auth/login sai mật khẩu — trả về 401 Unauthorized

FR-02 — Xoay vòng Token (Refresh Token)

Bước	Mô tả xử lý
1. Xuất phát	Client phát hiện AccessToken hết hạn (nhận lỗi 401), muốn lấy token mới mà không cần đăng nhập lại.
2. Gọi API	POST /api/v1/auth/refresh — Body JSON: { refreshToken: 'uuid...' }. Endpoint Public.
3. Xác thực	AuthController nhận @Valid @RequestBody RefreshTokenRequest — kiểm tra refreshToken không trống.
4. Xử lý Service	AuthServiceImpl.refreshToken(): (1) Tìm RefreshToken trong DB — không có ném 400. (2) Kiểm tra isRevoked — đã thu hồi ném 400. (3) Kiểm tra isExpired — hết hạn ném 400. (4) Thu hồi token cũ. (5) Tạo AccessToken mới + RefreshToken mới (Token Rotation).
5. Repository	RefreshTokenRepository.findByToken() tìm token. RefreshTokenRepository.save() thu hồi cũ và lưu mới.
6. Kết quả	HTTP 200 OK + { accessToken mới, refreshToken mới }



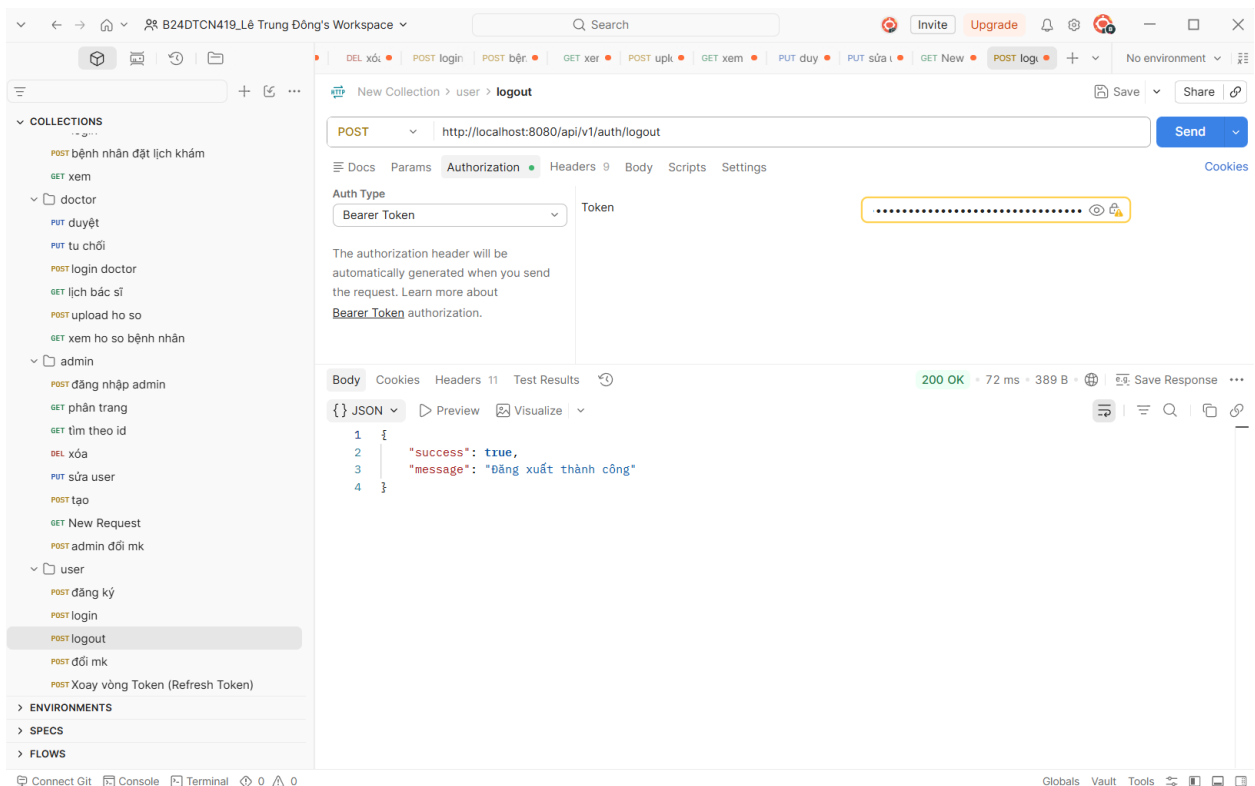
2.1 Kết quả POST /api/v1/auth/refresh thành công — trả về token mới



2.2 Kết quả khi refreshToken đã hết hạn hoặc bị thu hồi — trả về 400 Bad Request

FR-03 — Đăng xuất (Revoke Token)

Bước	Mô tả xử lý
1. Xuất phát	Người dùng đang đăng nhập muốn đăng xuất, vô hiệu hóa AccessToken hiện tại.
2. Gọi API	POST /api/v1/auth/logout — Header: Authorization: Bearer <accessToken>. Cần token hợp lệ.
3. Xác thực	JwtAuthenticationFilter chạy trước: xác thực JWT hợp lệ, kiểm tra token không nằm trong blacklist, chưa hết hạn. Không hợp lệ trả 401 trước khi vào Controller.
4. Xử lý Service	AuthServiceImpl.logout(accessToken): (1) Kiểm tra token không null. (2) JwtService.extractExpiration() lấy thời hạn của token. (3) Tạo TokenBlacklist entity lưu token + expiresAt vào DB. Từ đây mọi request dùng token này sẽ bị từ chối.
5. Repository	TokenBlacklistRepository.save() lưu token vào blacklist.
6. Kết quả	HTTP 200 OK + 'Đăng xuất thành công'. Mọi request tiếp theo dùng token cũ sẽ nhận 401.

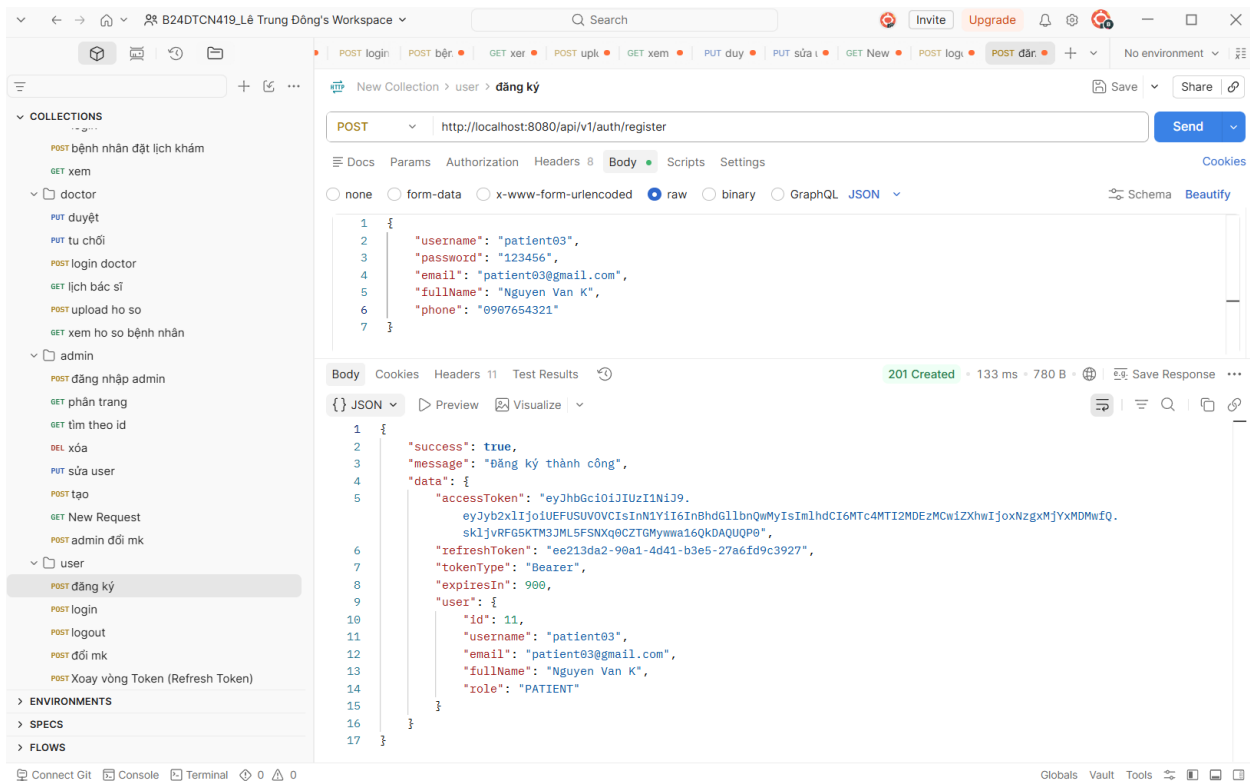


3.1 Kết quả POST /api/v1/auth/logout thành công — HTTP 200

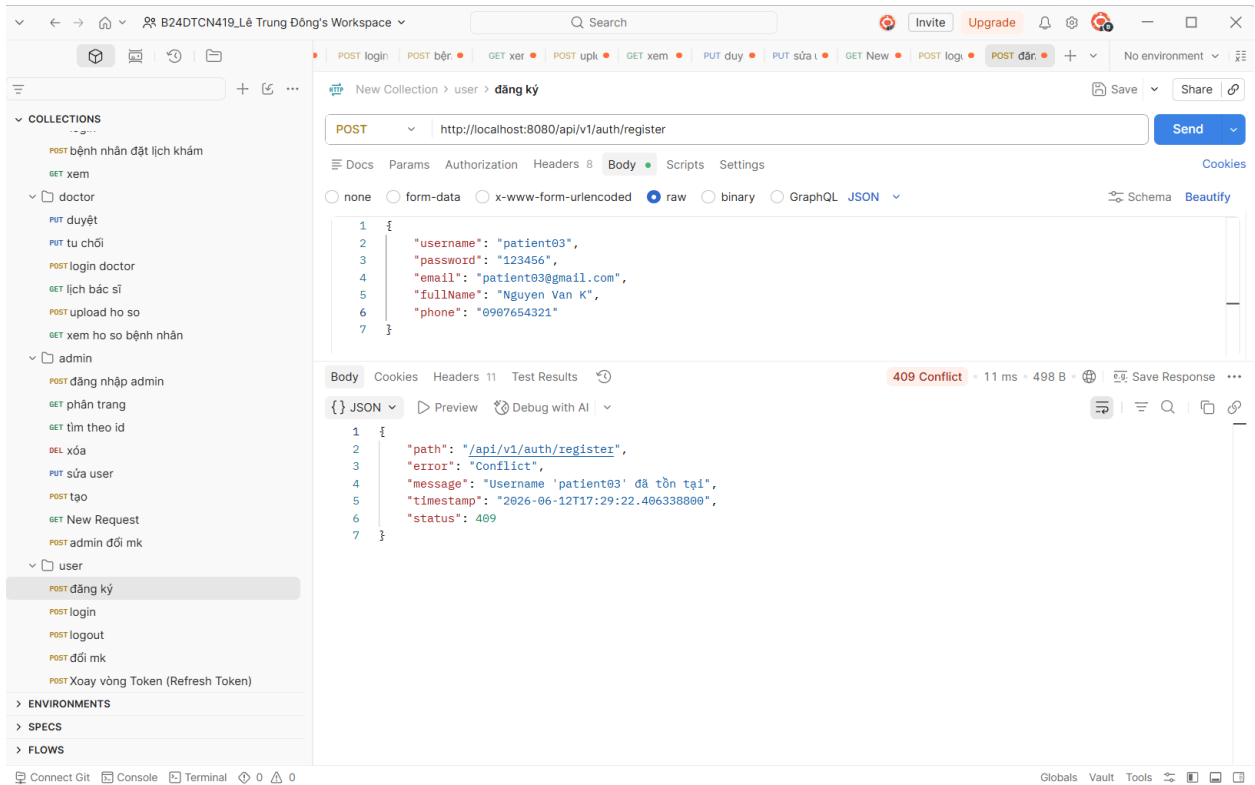
FR-04 — Đăng ký tài khoản Bệnh nhân mới

Bước	Mô tả xử lý
1. Xuất phát	Người dùng mới muốn tạo tài khoản bệnh nhân để sử dụng hệ thống đặt lịch khám.
2. Gọi API	POST /api/v1/auth/register — Body JSON: { username, password, email, fullName, phone }. Endpoint Public.
3. Xác thực	@Valid @RequestBody RegisterRequest kiểm tra: username, password, email, fullName không trống; email đúng định dạng; password đủ độ dài. Nếu sai trả về HTTP 400 kèm thông báo lỗi chi tiết.
4. Xử lý Service	AuthServiceImpl.register(): (1) userRepository.existsByUsername() — trùng ném 409 Conflict. (2)

	userRepository.existsByEmail() — trùng ném 409 Conflict. (3) Tạo User mới: role=PATIENT, status=ACTIVE, password mã hóa BCrypt strength 10. (4) Lưu user. (5) Tự động cấp AccessToken + RefreshToken.
5. Repository	UserRepository.existsByUsername(), existsByEmail() kiểm tra trùng. UserRepository.save() lưu user. RefreshTokenRepository.save() lưu token.
6. Kết quả	HTTP 201 Created + { accessToken, refreshToken, role: 'PATIENT', userInfo }



4.1 Kết quả POST /api/v1/auth/register thành công — HTTP 201 Created

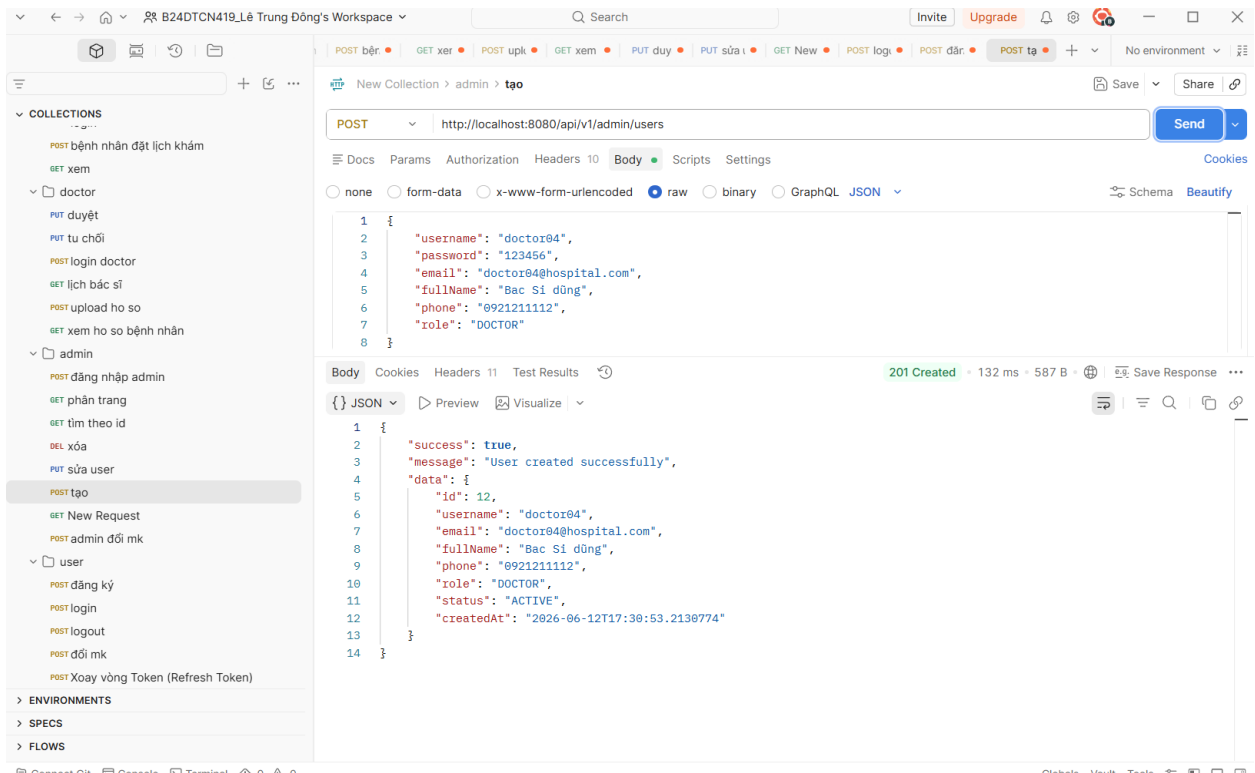


4.2 Kết quả đăng ký trùng username/email — HTTP 409 Conflict

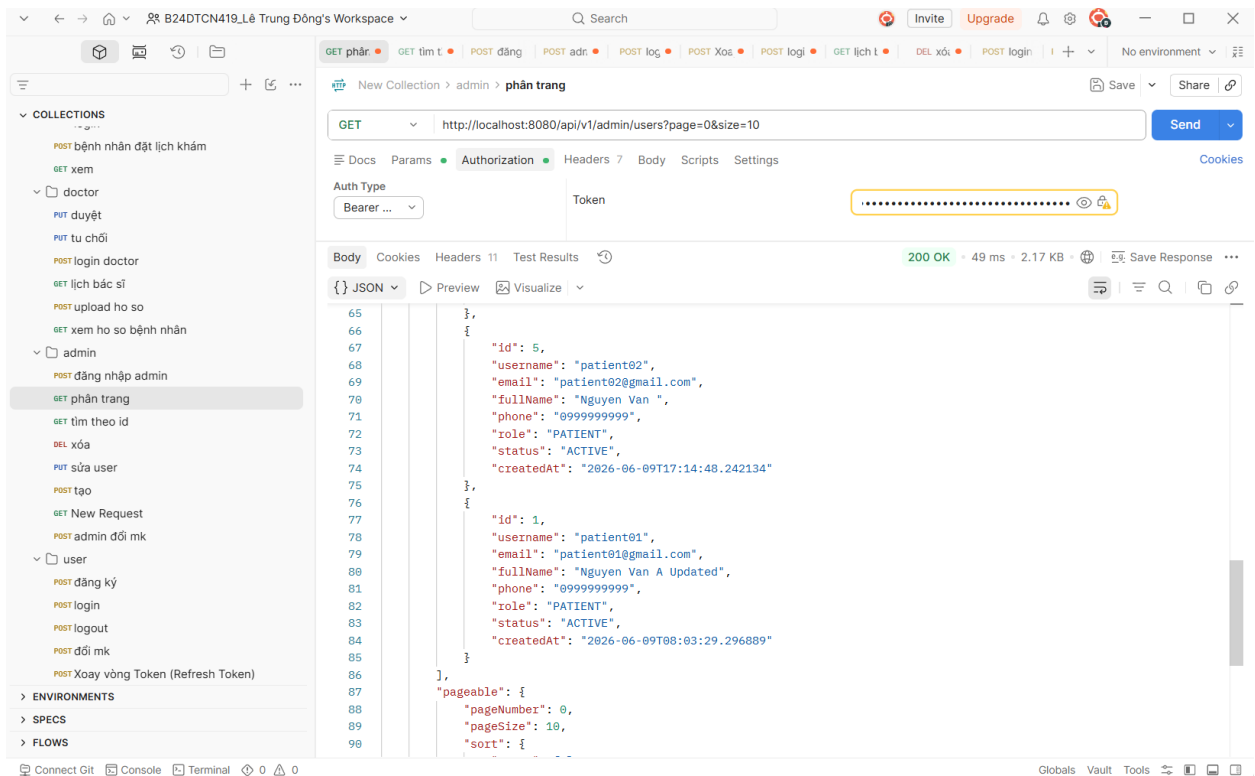
FR-05 — Quản lý Người dùng (CRUD, Tìm kiếm, Phân trang)

Bước	Mô tả xử lý
1. Xuất phát	Admin muốn tạo/xem/sửa/xóa tài khoản Doctor hoặc Patient trong hệ thống.
2. Gọi API	POST /api/v1/admin/users (tạo mới), GET /api/v1/admin/users?keyword=&role=&page=&size= (danh sách)... Tất cả cần Header: Authorization: Bearer <admin_token>.
3. Xác thực	SecurityConfig: /api/v1/admin/** yêu cầu ROLE_ADMIN — không đủ quyền trả 403 Forbidden. @Valid @RequestBody CreateUserRequest kiểm tra các trường bắt buộc.
4. Xử lý Service	UserServiceImpl: Create — kiểm tra trùng, BCrypt encode, lưu DB. Read List — searchUsers() dùng JPQL tìm theo tên/email/username, bắt buộc dùng

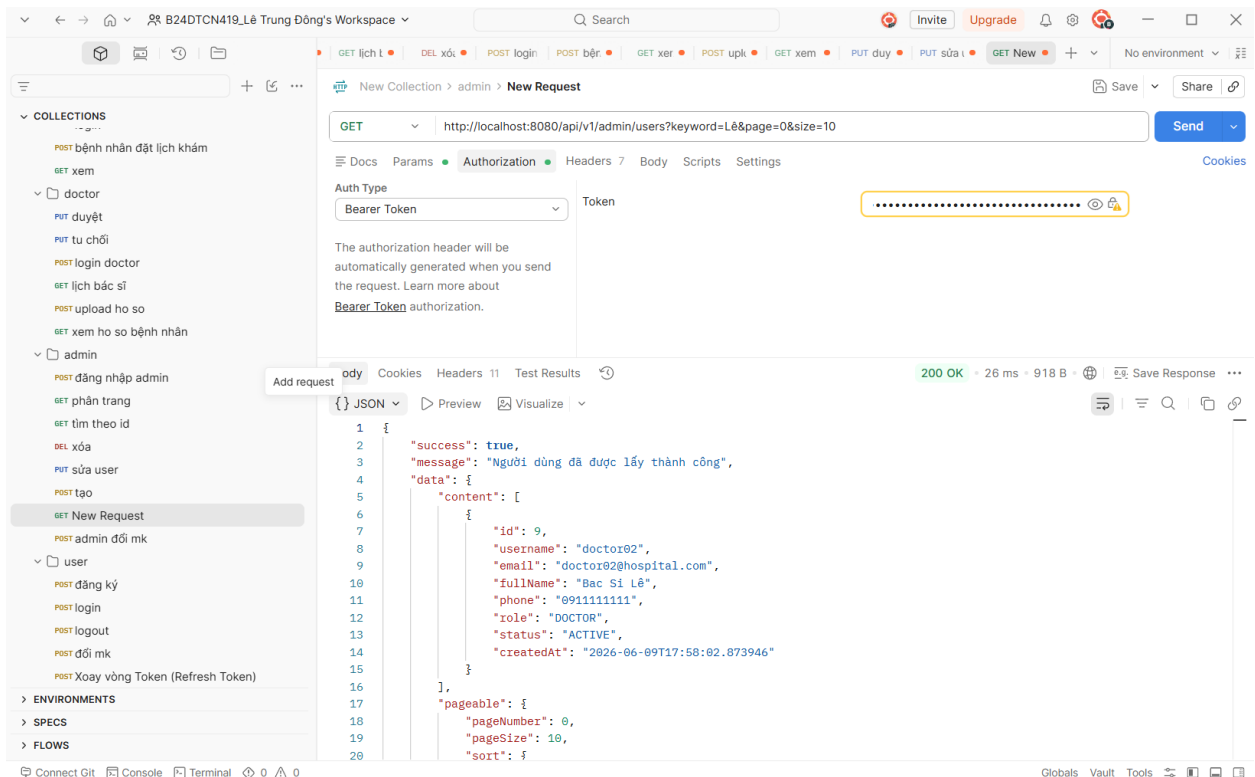
	Java Stream API để map Entity sang DTO. Read By ID — findById(). Update — cập nhật trường không null. Delete — kiểm tra không phải ADMIN rồi setStatus(INACTIVE) (soft delete).
5. Repository	UserRepository.searchUsers() JPQL tìm kiếm. UserRepository.findById(), save() thao tác DB.
6. Kết quả	Create: 201 + UserResponse. Read: 200 + Page<UserResponse> với phân trang. Update: 200 + UserResponse. Delete: 204 No Content.



5.1 POST /api/v1/admin/users — Tạo tài khoản DOCTOR mới thành công



5.2 GET /api/v1/admin/users?page=0&size=10 — Danh sách người dùng có phân trang



5.3 GET /api/v1/admin/users?keyword=nguyen&role=PATIENT — Tìm kiếm theo tên

The screenshot shows the Postman interface with a PUT request to `http://localhost:8080/api/v1/admin/users/5`. The request body is a JSON object: `{ "fullName": "tùng tùng", "phone": "0999999999", "status": "ACTIVE" }`. The response is a 200 OK status with a response time of 36 ms and a body of 596 B. The response body is a JSON object: `{ "success": true, "message": "Cập nhật người dùng thành công", "data": { "id": 5, "username": "patient02", "email": "patient02@gmail.com", "fullName": "tùng tùng", "phone": "0999999999", "role": "PATIENT", "status": "ACTIVE", "createdAt": "2026-06-09T17:14:48.242134" } }`.

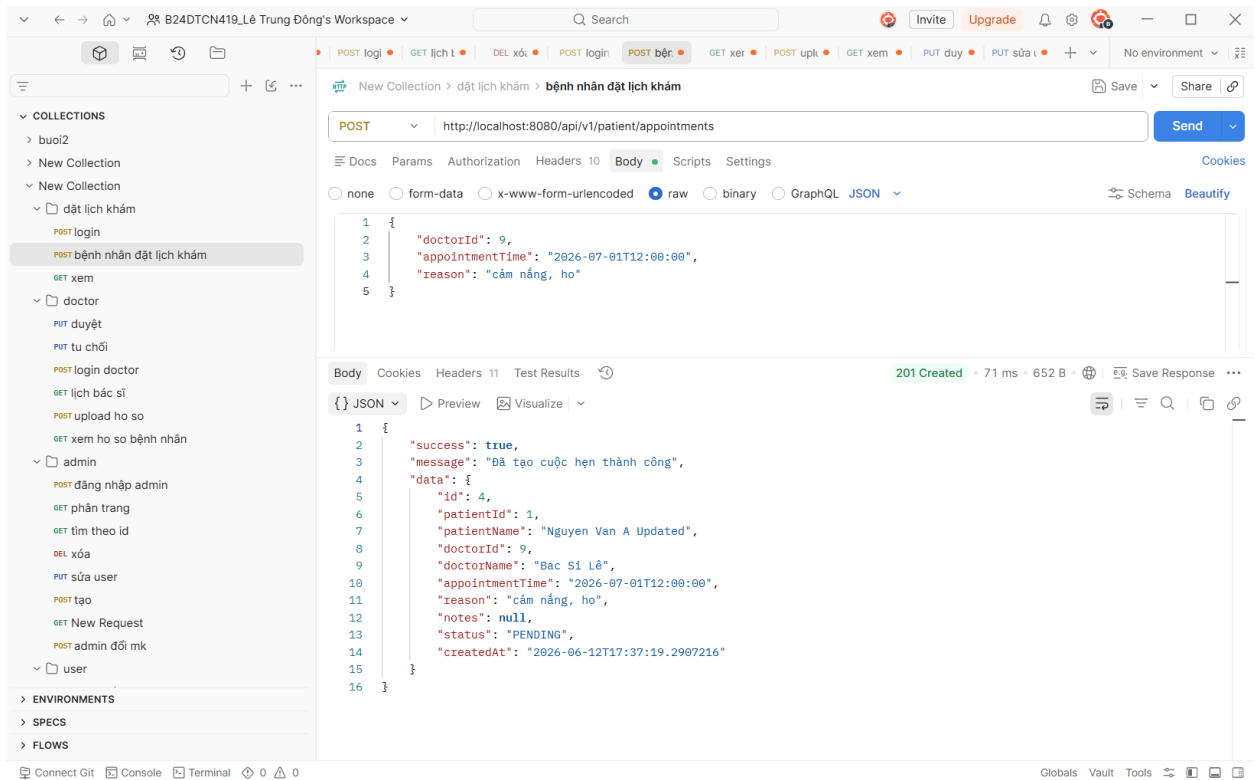
5.4 PUT /api/v1/admin/users/{id} — Cập nhật thông tin người dùng

The screenshot shows the Postman interface with a DELETE request to `http://localhost:8080/api/v1/admin/users/5`. The request is configured with Bearer Token authorization. The response is a 200 OK status with a response time of 28 ms and a body of 414 B. The response body is a JSON object: `{ "success": true, "message": "Người dùng đã hủy kích hoạt thành công" }`.

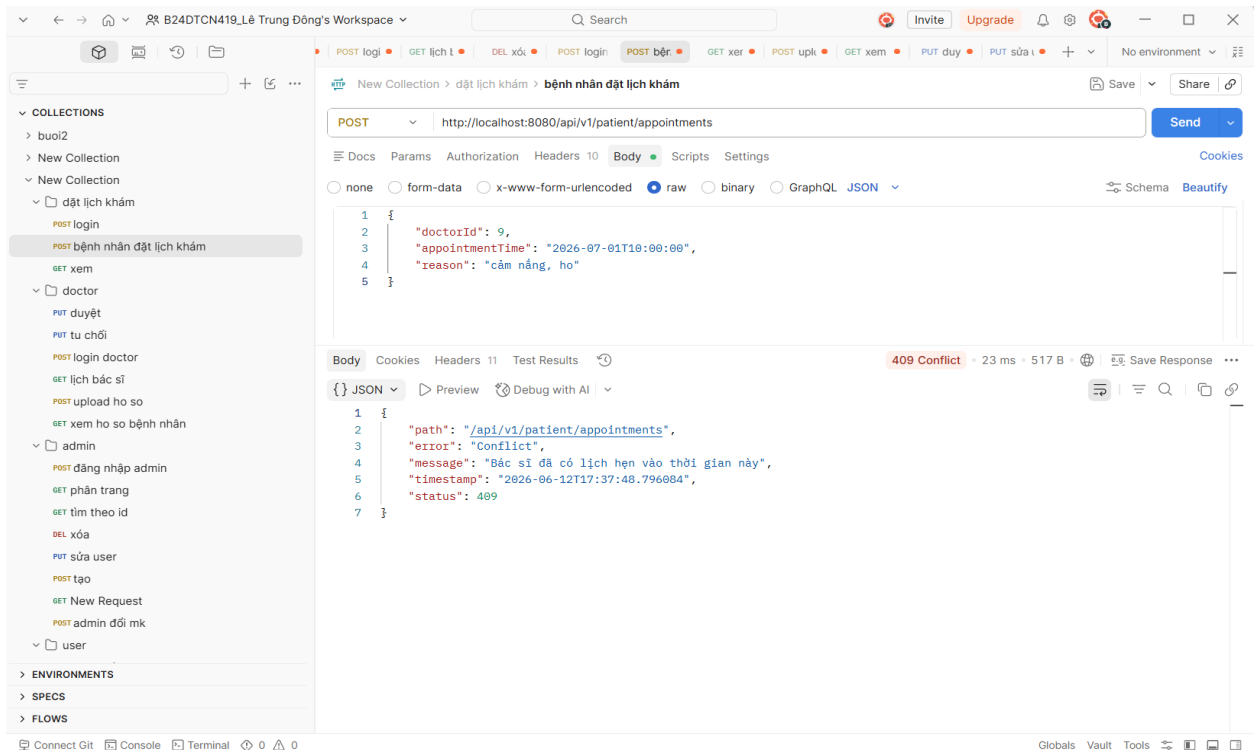
5.5 DELETE /api/v1/admin/users/{id} — Xóa (soft delete, đổi status INACTIVE)

FR-06 — Đặt lịch khám bệnh

Bước	Mô tả xử lý
1. Xuất phát	Bệnh nhân muốn đặt lịch khám với một bác sĩ cụ thể vào một thời điểm nhất định.
2. Gọi API	POST /api/v1/patient/appointments — Header: Authorization: Bearer <patient_token> — Body: { doctorId, appointmentTime, reason }.
3. Xác thực	SecurityConfig: /api/v1/patient/** yêu cầu ROLE_PATIENT. @Valid @RequestBody AppointmentRequest kiểm tra doctorId và appointmentTime không null.
4. Xử lý Service	AppointmentServiceImpl.createAppointment(): (1) Lấy username từ JWT qua authentication.getName(). (2) Tìm Patient và Doctor trong DB. (3) Kiểm tra doctor.getRole() == DOCTOR — sai ném 400. (4) Kiểm tra trùng lịch bác sĩ — trùng ném 409. (5) Tạo Appointment status=PENDING lưu DB. (6) AOP @AfterReturning tự động ghi log hệ thống.
5. Repository	UserRepository.findByUsername(), findById() lấy patient và doctor. AppointmentRepository.existsByDoctorAndAppointmentTimeAndStatusActive() kiểm tra trùng. AppointmentRepository.save() lưu lịch khám.
6. Kết quả	HTTP 201 Created + AppointmentResponse { id, patientName, doctorName, appointmentTime, status: 'PENDING', reason }



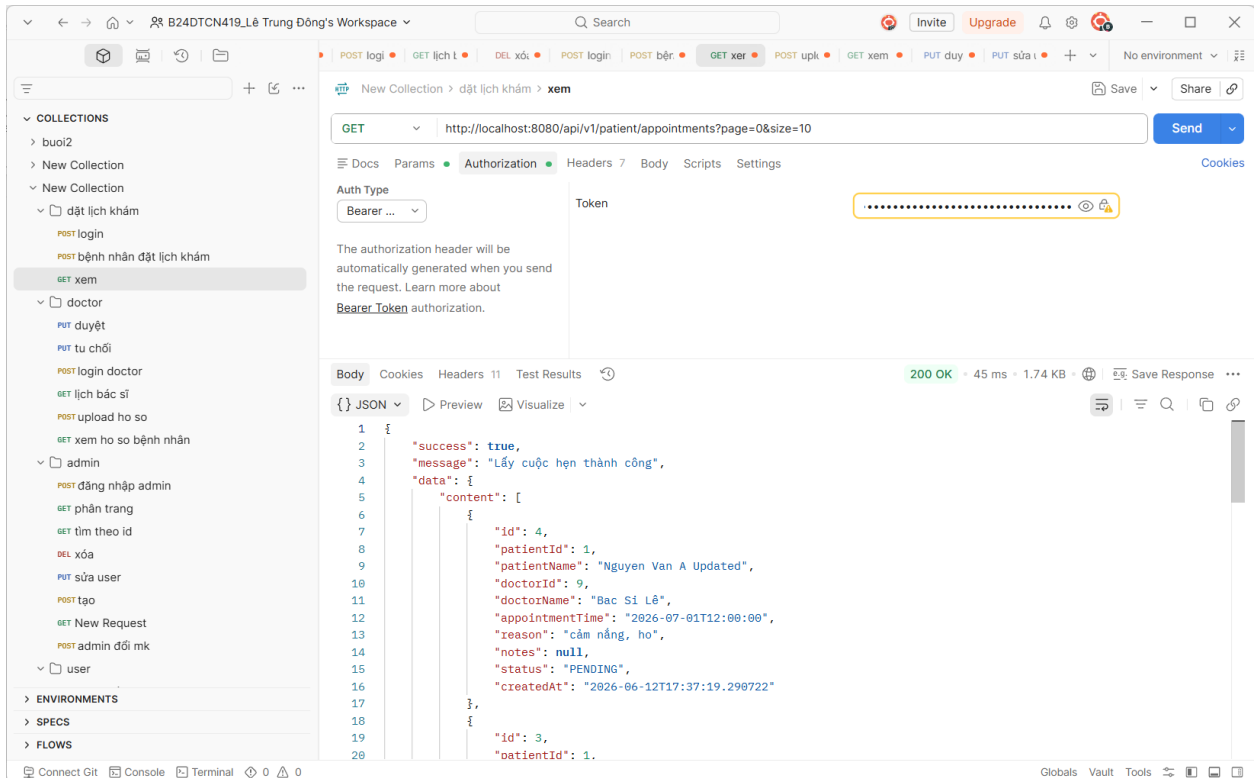
6.1 POST /api/v1/patient/appointments — Đặt lịch khám thành công, status PENDING



6.2 Kết quả đặt lịch trùng giờ của bác sĩ — HTTP 409 Conflict

FR-07 — Xem lịch sử khám bệnh cá nhân

Bước	Mô tả xử lý
1. Xuất phát	Bệnh nhân muốn xem toàn bộ lịch sử các lần đặt khám của mình, bao gồm trạng thái và thông tin bác sĩ.
2. Gọi API	GET /api/v1/patient/appointments?page=0&size=10 — Header: Authorization: Bearer <patient_token>. Không cần body.
3. Xác thực	SecurityConfig: ROLE_PATIENT. Username lấy từ JWT, không cần truyền thêm tham số nhận dạng.
4. Xử lý Service	AppointmentServiceImpl.getMyAppointments(): (1) Lấy username từ JWT. (2) Tìm patient. (3) findByPatientWithDetails() — JPQL dùng JOIN FETCH load patient và doctor trong 1 query tránh LazyInitializationException. (4) Stream API map Entity sang DTO.
5. Repository	UserRepository.findByUsername() lấy patient. AppointmentRepository.findByPatientWithDetails() lấy danh sách có phân trang, sắp xếp createdAt DESC.
6. Kết quả	HTTP 200 OK + Page<AppointmentResponse> với totalElements, totalPages, content là danh sách lịch khám.



7.1 GET /api/v1/patient/appointments — Danh sách lịch sử khám của bệnh nhân có phân trang

FR-08 — Phê duyệt / Từ chối lịch khám

Bước	Mô tả xử lý
1. Xuất phát	Bác sĩ xem danh sách lịch khám đang chờ duyệt (PENDING) và muốn phê duyệt (APPROVED) hoặc từ chối (REJECTED).
2. Gọi API	GET /api/v1/doctor/appointments?status=PENDING&page=0&size=10 (xem lịch). PUT /api/v1/doctor/appointments/{id}/status — Body: { status: 'APPROVED', notes: '...' }. Header cần token DOCTOR.
3. Xác thực	SecurityConfig: /api/v1/doctor/** yêu cầu ROLE_DOCTOR. @Valid @RequestBody AppointmentStatusRequest kiểm tra status không null.
4. Xử lý Service	Xem lịch: getDoctorAppointments() filter WHERE doctor = currentDoctor, chỉ thấy lịch của mình. Cập nhật: (1) Tìm appointment — không có 404. (2) Kiểm tra status == PENDING

	— không phải 400. (3) Kiểm tra status mới phải là APPROVED hoặc REJECTED. (4) Cập nhật status + notes lưu DB.
5. Repository	AppointmentRepository.findByDoctorWithDetails() lấy lịch của doctor. AppointmentRepository.findById() tìm appointment. AppointmentRepository.save() lưu trạng thái mới.
6. Kết quả	HTTP 200 OK + AppointmentResponse { status: 'APPROVED'/'REJECTED', notes }. State: PENDING → APPROVED hoặc PENDING → REJECTED.

The screenshot shows a Postman interface with a GET request to `http://localhost:8080/api/v1/doctor/appointments?status=PENDING&page=0&size=10`. The request is configured with Bearer Token authentication. The response is a 200 OK with a JSON body:

```

{
  "success": true,
  "message": "Success",
  "data": {
    "content": [
      {
        "id": 4,
        "patientId": 1,
        "patientName": "Nguyen Van A Updated",
        "doctorId": 9,
        "doctorName": "Bac Si Lê",
        "appointmentTime": "2026-07-01T12:00:00",
        "reason": "cảm nắng, ho",
        "notes": null,
        "status": "PENDING",
        "createdAt": "2026-06-12T17:37:19.290722"
      }
    ],
    "pageable": {
      "pageNumber": 0
    }
  }
}

```

8.1 GET /api/v1/doctor/appointments?status=PENDING — Danh sách lịch chờ duyệt của bác sĩ

The screenshot shows the Postman interface with a workspace named "B24DTCN419_Lê Trung Đông's Workspace". The left sidebar lists collections, with "doctor" expanded and "PUT duyệt" selected. The main panel shows a PUT request to `http://localhost:8080/api/v1/doctor/appointments/4/status`. The "Authorization" tab is active, showing "Bearer Token" authentication. The response body is a JSON object indicating success:

```

{
  "success": true,
  "message": "Tình trạng cuộc hẹn đã được cập nhật thành công",
  "data": {
    "id": 4,
    "patientId": 1,
    "patientName": "Nguyen Van A Updated",
    "doctorId": 9,
    "doctorName": "Bac Si Lê",
    "appointmentTime": "2026-07-01T12:00:00",
    "reason": "cảm nắng, ho",
    "notes": "Lịch khám đã được xác nhận",
    "status": "APPROVED",
    "createdAt": "2026-06-12T17:37:19.290722"
  }
}

```

The status bar at the bottom indicates a 200 OK response with a 29 ms latency and 701 B body size.

8.2 PUT /api/v1/doctor/appointments/{id}/status — Phê duyệt lịch khám, status APPROVED

The screenshot shows the Postman interface with the same workspace. The left sidebar shows "doctor" expanded and "PUT tu chối" selected. The main panel shows a PUT request to `http://localhost:8080/api/v1/doctor/appointments/5/status`. The "Authorization" tab is active, showing "Bearer Token" authentication. The response body is a JSON object indicating success:

```

{
  "success": true,
  "message": "Tình trạng cuộc hẹn đã được cập nhật thành công",
  "data": {
    "id": 5,
    "patientId": 1,
    "patientName": "Nguyen Van A Updated",
    "doctorId": 9,
    "doctorName": "Bac Si Lê",
    "appointmentTime": "2026-07-01T13:00:00",
    "reason": "cảm nắng, ho,choáng",
    "notes": "Bac si ban vào giờ này, vui lòng chọn giờ khác",
    "status": "REJECTED",
    "createdAt": "2026-06-12T17:45:53.270634"
  }
}

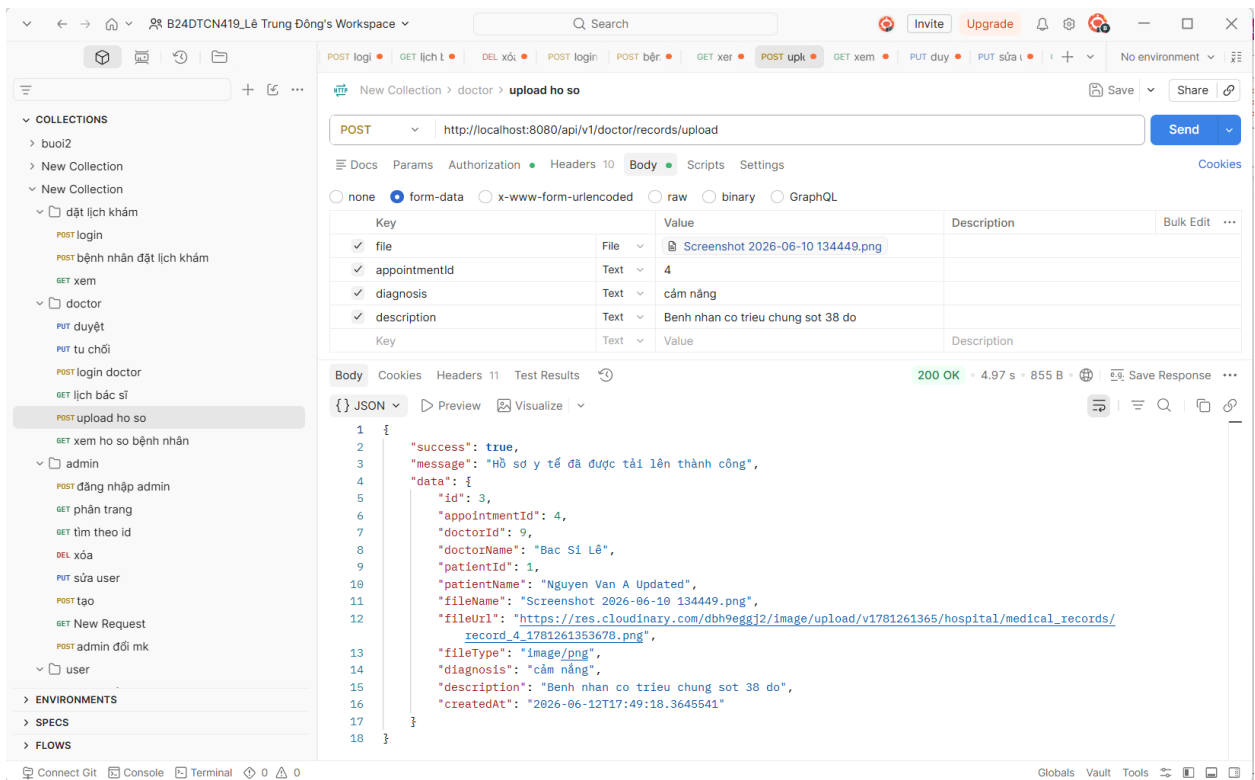
```

The status bar at the bottom indicates a 200 OK response with a 26 ms latency and 729 B body size.

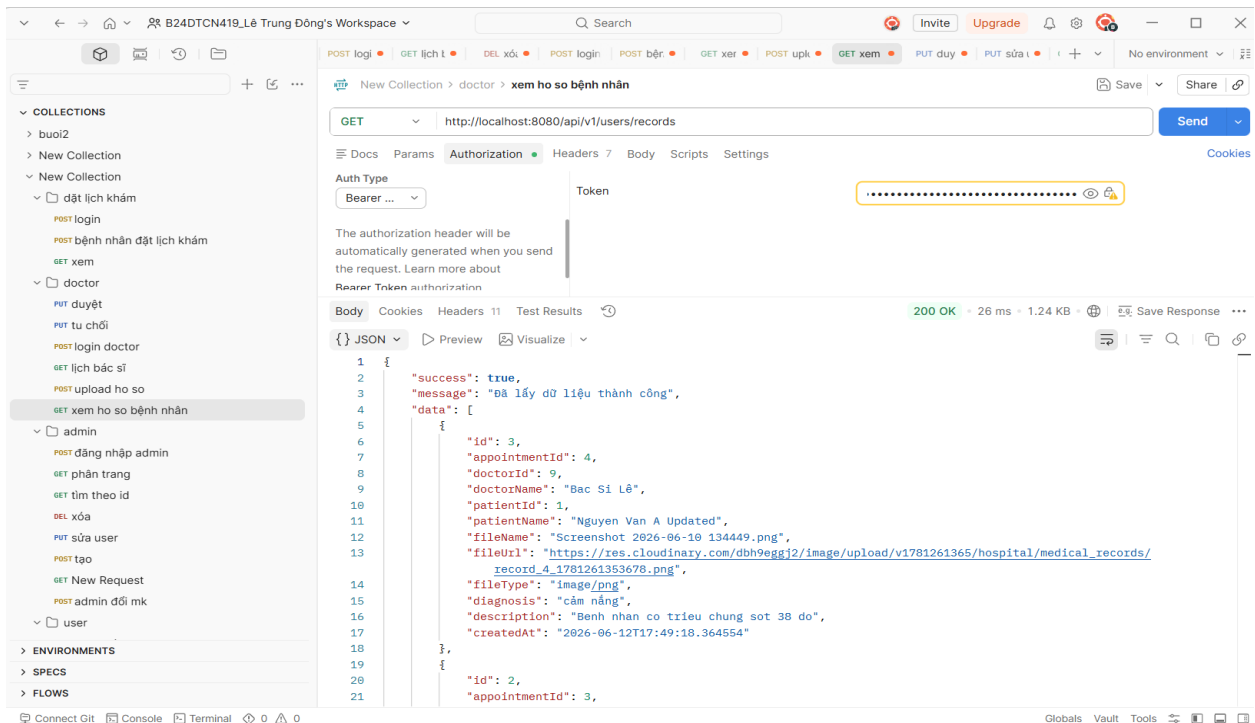
8.3 PUT /api/v1/doctor/appointments/{id}/status — Từ chối lịch khám, status REJECTED

FR-09 — Tải lên Hồ sơ bệnh án (Medical Record)

Bước	Mô tả xử lý
1. Xuất phát	Bác sĩ sau khi khám xong muốn tải lên hồ sơ bệnh án (ảnh siêu âm, kết quả xét nghiệm, đơn thuốc PDF) cho cuộc hẹn.
2. Gọi API	POST /api/v1/doctor/records/upload — Header: Authorization: Bearer <doctor_token> — Body: multipart/form-data với các trường: file (ảnh/pdf), appointmentId, diagnosis, description.
3. Xác thực	SecurityConfig: ROLE_DOCTOR. application.properties: max-file-size=10MB — vượt quá Spring tự reject 400. Kiểm tra file không rỗng và đúng định dạng.
4. Xử lý Service	MedicalRecordServiceImpl.uploadRecord(): (1) Xác thực doctor tồn tại. (2) Tìm Appointment theo appointmentId. (3) Gọi Cloudinary SDK: cloudinary.uploader().upload() — upload file lên cloud, nhận về secureUrl. (4) Tạo MedicalRecord { fileUrl: secureUrl, diagnosis, description } lưu DB.
5. Repository	UserRepository.findByUsername() xác thực doctor. AppointmentRepository.findById() tìm appointment. MedicalRecordRepository.save() lưu hồ sơ với URL Cloudinary.
6. Kết quả	HTTP 200 OK + MedicalRecordResponse { id, fileUrl (Cloudinary URL), diagnosis, description, appointmentId, doctorName }



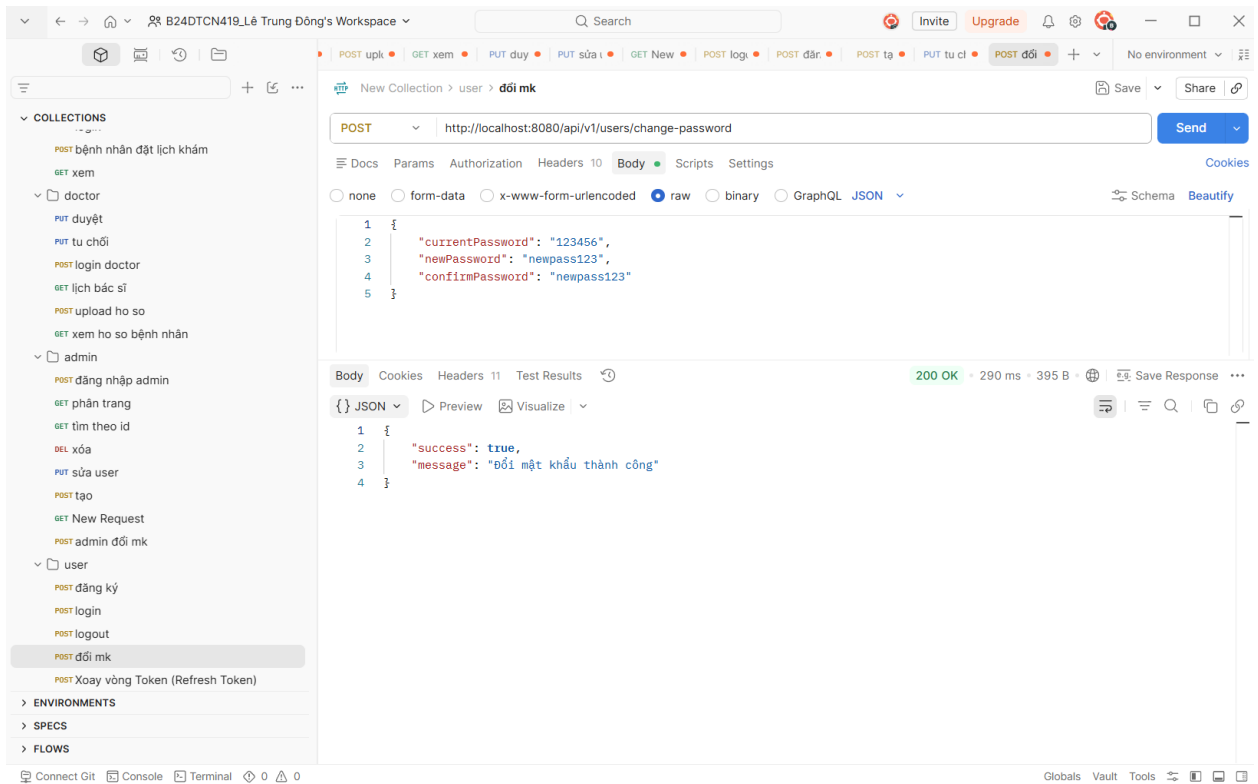
9.1 POST /api/v1/doctor/records/upload — Upload hồ sơ bệnh án, trả về fileUrl Cloudinary



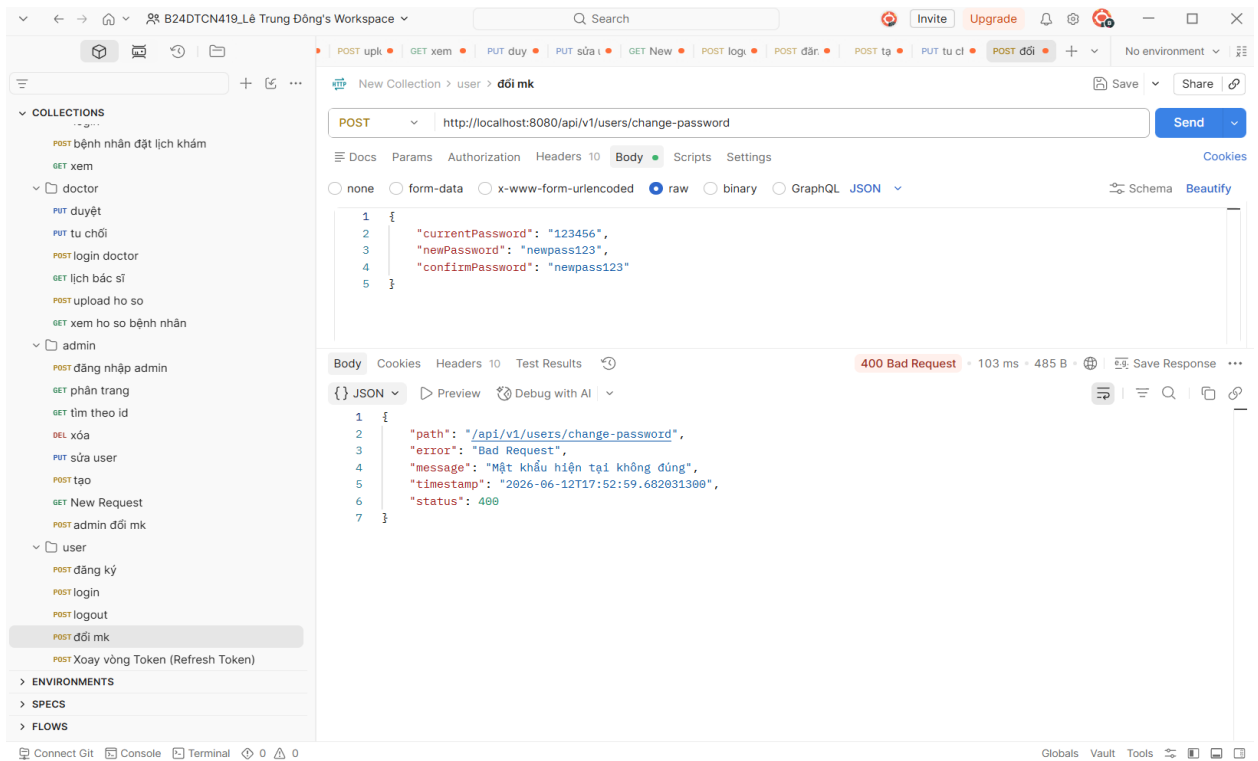
9.2 Xem tất cả các file mà bác sĩ đó đã khám đã được upload

FR-10 — Đổi mật khẩu

Bước	Mô tả xử lý
1. Xuất phát	Người dùng đã đăng nhập muốn thay đổi mật khẩu hiện tại của mình.
2. Gọi API	POST /api/v1/users/change-password — Header: Authorization: Bearer <token> — Body: { oldPassword, newPassword }.
3. Xác thực	JwtAuthenticationFilter: phải có token hợp lệ — không có 401. @Valid @RequestBody ChangePasswordRequest kiểm tra oldPassword và newPassword không trống, newPassword đủ độ dài tối thiểu.
4. Xử lý Service	UserServiceImpl.changePassword(): (1) Lấy username từ JWT qua authentication.getName(). (2) Tìm user trong DB. (3) passwordEncoder.matches(oldPassword, user.getPassword()) — sai ném 400. (4) Kiểm tra newPassword khác oldPassword. (5) BCrypt encode newPassword rồi setPassword() lưu DB.
5. Repository	UserRepository.findByUsername() tìm user. UserRepository.save() lưu mật khẩu mới đã mã hóa BCrypt.
6. Kết quả	HTTP 200 OK + 'Đổi mật khẩu thành công'. Lần đăng nhập tiếp theo phải dùng mật khẩu mới.



10.1 POST /api/v1/users/change-password — Đổi mật khẩu thành công HTTP 200



10.2 Kết quả nhập sai mật khẩu cũ — HTTP 400 Bad Request

FR-11 — Ghi log thời gian thực hiện (Advanced Feature 1)

Bước	Mô tả xử lý
1. Xuất phát	Client thực hiện bất kỳ yêu cầu HTTP Request nào đến các endpoint thuộc tầng Controller hoặc tầng Service.
2. Gọi API	Tất cả các API khả dụng trong hệ thống (Public hoặc Authenticated) đều nằm trong vùng quét khía cạnh (Pointcut).
3. Xác thực	Aspect-Oriented Programming (Spring AOP) tự động đánh chặn trước khi yêu cầu đi vào bên trong hàm nghiệp vụ xử lý chính.
4. Xử lý Service	Khía cạnh LoggingAspect thực thi advice dạng @Around: (1) Lưu mốc thời gian bắt đầu thực thi thông qua System.currentTimeMillis(). (2) Cho phép hàm chính chạy tiếp bằng joinPoint.proceed(). (3) Lắng nghe nếu có lỗi hoặc hoàn thành suôn sẻ. (4) Tính tổng thời gian chạy và in trực tiếp ra bảng điều khiển hệ thống.
5. Repository	Không tác động trực tiếp lên DB. Sử dụng thư viện Slf4j Logger để xuất dữ liệu text log nhanh chóng.
6. Kết quả	Console Log xuất định dạng chuẩn: [PERFORMANCE LOG] Phương thức [Tên_Class].[Tên_Hàm]() thực hiện thành công trong: [X] ms

```
Run HospitalManagementApplication x
Console Beans Health Mappings Environment
ent] [nio-8080-exec-2] c.e.h.aspect.LoggingAspect : [AOP] --> Entering: UserServiceImpl.changePassword() with args: [patient03, ChangePasswordRequest(current
ent] [nio-8080-exec-2] c.e.h.aspect.LoggingAspect : [EXCEPTION] UserServiceImpl.changePassword() threw BadRequestException: Mật khẩu hiện tại không đúng
ent] [nio-8080-exec-2] c.e.h.aspect.LoggingAspect : [PERF] UserServiceImpl.changePassword() failed after 86 ms - Error: Mật khẩu hiện tại không đúng
ent] [nio-8080-exec-2] c.e.h.aspect.LoggingAspect : [PERF] UserController.changePassword() failed after 91 ms - Error: Mật khẩu hiện tại không đúng
ent] [nio-8080-exec-2] .m.m.s.ExceptionHandlerExceptionHandlerResolver : Resolved [com.example.hospitalmanagement.exception.BadRequestException: Mật khẩu hiện tại không đúng]
```

11.1 Kết quả in log hiệu năng tự động trên Console IntelliJ IDEA

FR-12 — Cài đặt hệ thống kiểm thử tự động Unit Test (Advanced Feature 2)

Bước	Mô tả xử lý
1. Xuất phát	Lập trình viên chạy lệnh kiểm thử (mvn test) hoặc chạy từng file test cụ thể để đánh giá độ ổn định Backend.
2. Gọi API	Chạy nội bộ trong môi trường JUnit 5 + Mockito framework, không cần khởi động toàn bộ server Tomcat.
3. Xác thực	Mô phỏng dữ liệu giả định: @Mock cho tầng Repositories để test tầng Service; Sử dụng MockMvc để kiểm tra tầng Controllers.
4. Xử lý Service	Thực thi kiểm tra bao phủ 10 ca kiểm thử (5 Service Cases cho UserServiceImpl và 5 Controller Cases cho AuthController). Đảm bảo mọi nhánh logic (Thành công, Sai mật khẩu, Trùng bản ghi, Thiếu dữ liệu) đều trả về HTTP status và Exception mong muốn.
5. Repository	Sử dụng Mockito.when(...).thenReturn(...) nhằm cố lập hoàn toàn cơ sở dữ liệu vật lý MySQL.

- Security Layer: JWT filter, xác thực và phân quyền
- Aspect Layer: AOP logging tự động, tách biệt khỏi business logic

3.2. Cấu trúc thư mục

```

com.example.hospitalmanagement
├── aspect/           LoggingAspect.java (AOP)
├── config/           SecurityConfig, CloudinaryConfig,
ApplicationConfig
├── controller/       AuthController, AdminController,
DoctorController, PatientController, UserController
├── dto/
│   ├── request/      LoginRequest, RegisterRequest, AppointmentRequest...
│   └── response/      AuthResponse, AppointmentResponse,
UserResponse...
├── entity/           User, Appointment, MedicalRecord, RefreshToken...
├── exception/        GlobalExceptionHandler, BadRequestException...
├── repository/       UserRepository, AppointmentRepository...
├── security/         JwtAuthenticationFilter, JwtService
└── service/          AuthService, UserService, AppointmentService...
    └── impl/         AuthServiceImpl, UserServiceImpl...

```

3.3. Ma trận phân quyền API

API Endpoint	ADMIN	DOCTOR	PATIENT	PUBLIC
/api/v1/auth/**	✓	✓	✓	✓
/api/v1/admin/**	✓	X	X	X
/api/v1/doctor/**	X	✓	X	X
/api/v1/patient/**	X	X	✓	X
/api/v1/users/**	✓	✓	✓	X

IV. CƠ CHẾ BẢO MẬT & CHỨC NĂNG NÂNG CAO

4.1. Cơ chế JWT ba tầng

- AccessToken: JWT có thời hạn 15 phút, gửi kèm trong Header mỗi request (Authorization: Bearer <token>).
- RefreshToken: UUID ngẫu nhiên, thời hạn 7 ngày. Dùng để xin cấp AccessToken mới mà không cần đăng nhập lại.
- TokenBlacklist: Khi logout, AccessToken được lưu vào blacklist. JwtAuthenticationFilter kiểm tra blacklist trước khi chấp nhận request.

4.2. BCrypt Password Encoding

Mật khẩu không bao giờ lưu dưới dạng plaintext. Hệ thống dùng BCrypt với strength=10 để mã hóa một chiều. Khi đăng nhập, Spring Security dùng passwordEncoder.matches() để so sánh.

4.3. Environment Variables

Các thông tin nhạy cảm được đưa ra biến môi trường, không hardcode trong code:

- jwt.secret-key=\${JWT_SECRET:default_value}
- clouinary.api-secret=\${CLOUDINARY_API_SECRET:default_value}
- spring.datasource.password=\${DB_PASSWORD:default_value}

4.4. Chi tiết cơ chế mã nguồn FR-11 (Spring AOP Logging)

```
@Aspect
@Component
@Slf4j
public class LoggingAspect {
    @Pointcut("execution(* com.example.hospitalmanagement.controller..*.*(..)) ||
execution(* com.example.hospitalmanagement.service..*.*(..)")
    public void applicationPackagePointcut() {}

    @Around("applicationPackagePointcut()")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable
    {
        long startTime = System.currentTimeMillis();
        String className = joinPoint.getTarget().getClass().getSimpleName();
        String methodName = joinPoint.getSignature().getName();
        try {
            Object result = joinPoint.proceed();
            long executionTime = System.currentTimeMillis() - startTime;
            log.info("[PERFORMANCE LOG] Phương thức {}.{}() thực hiện thành công
trong: {} ms", className, methodName, executionTime);
            return result;
        } catch (Throwable throwable) {
```

```

        long executionTime = System.currentTimeMillis() - startTime;
        log.error("[PERFORMANCE LOG] Phương thức {}.{}() thất bại sau: {} ms.
Nguyên nhân: {}", className, methodName, executionTime, throwable.getMessage());
        throw throwable;
    }
}
}

```

4.5. Chi tiết các ca kiểm thử FR-12 (Unit Test)

Hệ thống kiểm thử bao gồm các kịch bản chặt chẽ cho tầng Service (kiểm tra logic nghiệp vụ cấu trúc đổi mật khẩu, xử lý ngoại lệ) và tầng Controller (sử dụng MockMvc kiểm tra mã phản hồi HTTP trạng thái đầu vào).

Tất cả 10 ca kiểm thử đều được cấu trúc chạy tự động, cô lập dữ liệu hoàn toàn để phục vụ quá trình CI/CD tích hợp liên tục.

V. KẾT LUẬN

5.1. Kết quả đạt được

Dự án đã triển khai đầy đủ 12 chức năng theo yêu cầu SRS với kiến trúc layered chuẩn Spring Boot:

- FR-01 đến FR-04: Hoàn thiện hệ thống xác thực JWT ba tầng với AccessToken, RefreshToken và TokenBlacklist.
- FR-05: Quản lý người dùng đầy đủ CRUD với tìm kiếm và phân trang, bắt buộc dùng Java Stream API.
- FR-06 đến FR-08: Quy trình đặt lịch khám hoàn chỉnh từ bệnh nhân đặt → bác sĩ duyệt/từ chối.
- FR-09: Tích hợp Cloudinary upload file hồ sơ bệnh án lên đám mây.
- FR-10: Chức năng đổi mật khẩu an toàn với BCrypt.
- FR-11 (AF1): Ứng dụng khía cạnh Spring AOP tự động đo lường và ghi log hiệu năng thực thi của các phương thức hệ thống.
- FR-12 (AF2): Thiết lập hệ thống kiểm thử tự động với 10 Unit Tests (Service & Controller) đạt tỷ lệ passed 100%, bảo đảm mã nguồn Backend vững chắc.
- Soft Delete: Không xóa dữ liệu khỏi DB, chỉ đổi status INACTIVE để bảo toàn lịch sử.

5.2. Hướng phát triển

- Bổ sung chức năng quên mật khẩu qua email (Spring Mail).
- Thêm @Scheduled task để tự động dọn dẹp token hết hạn trong bảng token_blacklist.
- Triển khai Docker để đóng gói toàn bộ ứng dụng Backend và cơ sở dữ liệu MySQL giúp đồng bộ hóa quy trình triển khai lên đám mây AWS/GCP.